Trabajo Práctico 5 – Ejercicios extra

SISTEMAS OPERATIVOS

Tracks

1. Límite de instancias con un semáforo (7/10) **3**

2. Aproximación de π y rendimiento (5/10) **4**

3. Transferencia por pipes y SSH (4/10) **5**

4. Cliente-servidor: solicitud y respuesta (6/10) **6**

5. Heartbeat y watchdog (6/10) **7**

6. Comunicación entre pares con FIFOs nombradas (5/10) **8**

1. Límite de instancias con un semáforo (7/10)

TRACK – 7/10

Descripción: limitar la cantidad de instancias concurrentes de un programa usando un semáforo contador de System V.

Implementá un programa que limite la cantidad de instancias concurrentes usando un semáforo de System V. Interfaz sugerida:

```
instance-limit SEM_KEY MAX_INSTANCES -- COMMAND [ARGS...]
```

Investigá la inicialización, la reserva y liberación de cupos, el comportamiento cuando se alcanza el límite y la limpieza del semáforo.

2. Aproximación de π y rendimiento (5/10)

TRACK – 5/10

Descripción: aproximar π sumando términos de una expansión y medir el costo de aumentar la cantidad de términos.

Implementá dos versiones de un programa que reciba N y calcule:

$$\pi_N = 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1}.$$

Una versión debe ser secuencial. La otra debe repartir los términos entre varios trabajadores concurrentes, recibir una cantidad configurable de trabajadores y combinar los resultados parciales.

El error de esta aproximación cumple

$$|\pi - \pi_N| < \frac{4}{2N+1}.$$

Mostrá la aproximación, el error y el tiempo de ejecución de ambas versiones. Probá distintos valores de N y distintas cantidades de trabajadores; compará los resultados y el costo de la coordinación.

BONUS – POR PUNTOS

Conservá las versiones secuencial y multi-worker de la consigna y agregá una tercera versión usando OpenMP. Las tres deben calcular la misma aproximación, aceptar parámetros equivalentes y producir resultados comparables. Compará las tres ejecuciones con distintas cantidades de trabajadores o hilos e informá el speedup. Se sugiere usar C o C++.

3. Transferencia por pipes y SSH (4/10)

TRACK – 4/10

Descripción: transferir y extraer un directorio remoto usando un pipe, sin crear un archivo intermedio.

Investigá brevemente qué resuelven `scp` y `rsync`, y compará esas herramientas con una transferencia basada en un flujo de datos.

Después, escribí un `one-liner` que:

- empaquete un directorio local y envíe el resultado por la salida estándar;
- use `ssh` para transportar ese flujo;
- extraiga el flujo en un directorio de la máquina remota.

La solución debe usar pipes, `ssh` y `tar`. No uses `scp` ni `rsync` para resolverla, y no generes un archivo intermedio. Probá la línea con una máquina remota y verificá que la estructura y el contenido lleguen correctamente.

4. Cliente-servidor: solicitud y respuesta (6/10)

TRACK – 6/10

Descripción: implementar una comunicación básica entre un proceso cliente y un proceso servidor.

Implementá dos procesos:

- el servidor espera una solicitud, la procesa y envía una respuesta;
- el cliente envía la solicitud y muestra la respuesta recibida.

Elegí una operación sencilla, por ejemplo, solicitar al servidor que tire un dado o enviarle una cuenta aritmética para que devuelva el resultado. Definí un protocolo simple y asegurá al menos un intercambio completo de solicitud y respuesta mediante un mecanismo de comunicación entre procesos.

BONUS – POR PUNTOS

Convertí la solución en un juego mínimo con dos procesos. El cliente debe mostrar un rectángulo grande usando secuencias de escape ANSI y un personaje dentro del área. Configurá la entrada del terminal sin modo canónico ni eco para leer las teclas `W`, `A`, `S` y `D` inmediatamente, y restaurá la configuración al salir. Como referencia para las secuencias de control del terminal, consultá Xterm Control Sequences.
Cada tecla debe enviarse al servidor. El servidor debe validar y aplicar el movimiento, devolver el estado actualizado, incluyendo la posición del personaje, y mantener la autoridad exclusiva sobre el estado. El cliente debe renderizar en tiempo real únicamente el estado recibido.

5. Heartbeat y watchdog (6/10)

TRACK – 6/10

Descripción: detectar la ausencia de un proceso mediante mensajes periódicos de heartbeat.

Implementá dos procesos:

- un proceso de trabajo que envíe periódicamente un mensaje de heartbeat;
- un watchdog que reciba esos mensajes y controle el tiempo transcurrido desde el último heartbeat.

Si el watchdog no recibe un heartbeat dentro de un timeout configurable, debe informar que el proceso dejó de responder. Usá un mecanismo de comunicación entre procesos, definí el formato de los mensajes y asegurá el cierre correcto de los descriptores.

Probá tanto el funcionamiento normal como la ausencia o terminación del proceso de trabajo.

6. Comunicación entre pares con FIFOs nombradas (5/10)

TRACK – 5/10

Descripción: construir una comunicación local entre procesos pares usando FIFOs (*First In, First Out*) nombradas.

Construí una aplicación de mensajería local. Una sala se representa con una carpeta compartida y cada proceso crea dentro de ella su propia FIFO nombrada. Los procesos deben descubrirse y enviarse mensajes directamente, sin un servidor central. Definí un protocolo simple, separá los mensajes sucesivos y limpiá la FIFO al salir. Podés usar Protocol Buffers (protobuf) para serializar los mensajes.

Probá varios pares en una misma sala, la entrada y salida de procesos y la desaparición inesperada de uno de ellos. Observá las FIFOs y sus descriptores con `ls -l` y `/proc/<pid>/fd`.

BONUS – POR PUNTOS

Convertí la aplicación en una sala de chat local entre pares. Agregá una interfaz de terminal cómoda que permita escribir mensajes y mostrar los mensajes entrantes, manteniendo la comunicación directa, las salas por carpetas y una salida limpia. Podés consultar la documentación Xterm Control Sequences para las secuencias de control del terminal.